Tutorial --- Construção do banco de dados `escola` para conexão com Java

1. Objetivo

Este tutorial reúne a etapa de criação do banco de dados MySQL apresentada na Aula 07 --- Banco de dados + Java. A finalidade é deixar o banco `escola` pronto para ser utilizado por uma aplicação Java Swing.

A sequência é:

1. Abrir o MySQL.
2. Criar e selecionar o banco `escola`.
3. Criar as tabelas `alunos`, `professores` e `matriculas`.
4. Inserir registros de teste.
5. Conferir as tabelas e consultar os dados.
6. Utilizar as informações de conexão no projeto Java.

Observação: Os nomes das tabelas e os campos abaixo seguem os exercícios da Aula
Observação: 07. Os registros inseridos são exemplos fictícios para testar o banco.

2. Abrir o MySQL

Na aula, o professor utiliza o MySQL Command Line Client. Também é possível utilizar uma ferramenta como XAMPP, WAMP ou outra instalação MySQL.

1. Abra o MySQL Command Line Client.
2. Informe a senha do usuário `root`.
3. Depois de entrar, os comandos SQL poderão ser executados.

A senha `1234` e a senha `laboratorio` aparecem no material apenas como exemplos do ambiente utilizado na aula. Utilize a senha configurada na sua instalação.

3. Verificar os bancos existentes

```
SHOW DATABASES;
```

Esse comando lista os bancos de dados disponíveis no servidor MySQL.

4. Criar o banco `escola`

```
CREATE DATABASE escola;
```

O comando cria o banco de dados com o nome `escola`.

5. Selecionar o banco

Antes de criar as tabelas, selecione o banco que será utilizado:

```
USE escola;
```

A partir desse momento, os comandos de criação de tabelas serão executados dentro do banco `escola`.

6. Criar a tabela `alunos`

De acordo com o exercício da Aula 07, a tabela deve possuir:

- `id`: chave primária com incremento automático.
- `nome`: texto de até 50 caracteres.
- `idade`: número inteiro.
- `curso`: texto de até 50 caracteres.

```
CREATE TABLE alunos (
    id INT NOT NULL AUTO_INCREMENT,
    nome VARCHAR(50),
    idade INT,
    curso VARCHAR(50),
    PRIMARY KEY (id)
);
```

7. Criar a tabela `professores`

Campos solicitados no exercício:

- `id`: chave primária com incremento automático.
- `nome`: texto de até 50 caracteres.
- `idade`: número inteiro.
- `disciplina`: texto de até 50 caracteres.

```
CREATE TABLE professores (
    id INT NOT NULL AUTO_INCREMENT,
    nome VARCHAR(50),
    idade INT,
    disciplina VARCHAR(50),
    PRIMARY KEY (id)
```

```
);
```

8. Criar a tabela `matriculas`

A tabela de matrículas relaciona alunos e professores. Os campos solicitados são:

- `id`: chave primária com incremento automático.
- `id_aluno`: chave estrangeira que referencia `alunos`.
- `id_professor`: chave estrangeira que referencia `professores`.
- `data_matricula`: data da matrícula.

```
CREATE TABLE matriculas (
    id INT NOT NULL AUTO_INCREMENT,
    id_aluno INT,
    id_professor INT,
    data_matricula DATE,
    PRIMARY KEY (id),
    FOREIGN KEY (id_aluno) REFERENCES alunos(id),
    FOREIGN KEY (id_professor) REFERENCES professores(id)
);
```

As chaves estrangeiras fazem a ligação entre os registros de matrículas, alunos e professores. Por isso, os alunos e professores relacionados precisam existir antes da inserção da matrícula.

9. Conferir a criação das tabelas

Para listar as tabelas do banco selecionado:

```
SHOW TABLES;
```

Para visualizar a estrutura de cada tabela:

```
DESC alunos;
DESC professores;
DESC matriculas;
```

Confira se os nomes e tipos dos campos estão corretos.

10. Inserir alunos de teste

O exercício solicita cinco registros. Abaixo estão cinco exemplos fictícios:

```
INSERT INTO alunos (nome, idade, curso) VALUES
('Ana Souza', 20, 'Ciência da Computação'),
('Bruno Lima', 22, 'Sistemas de Informação'),
('Carla Mendes', 19, 'Administração'),
('Daniel Rocha', 21, 'Ciência da Computação'),
('Eduarda Alves', 23, 'Engenharia de Software');
```

11. Inserir professores de teste

O exercício solicita três registros. Abaixo estão três exemplos fictícios:

```
INSERT INTO professores (nome, idade, disciplina) VALUES
('Marcos Silva', 40, 'Banco de Dados'),
('Juliana Costa', 35, 'Programação'),
('Pedro Oliveira', 45, 'Engenharia de Software');
```

12. Inserir matrículas de teste

O exercício solicita cinco matrículas relacionando alunos e professores:

```
INSERT INTO matriculas (id_aluno, id_professor, data_matricula) VALUES
(1, 1, '2026-03-10'),
(2, 2, '2026-03-11'),
(3, 3, '2026-03-12'),
(4, 1, '2026-03-13'),
(5, 2, '2026-03-14');
```

Os números utilizados em `id_aluno` e `id_professor` correspondem aos IDs gerados para os registros de exemplo. Se o banco já possuir dados, confira os IDs antes de executar esse comando.

13. Consultar os dados

Para visualizar todos os alunos:

```
SELECT * FROM alunos;
```

Para visualizar todos os professores:

```
SELECT * FROM professores;
```

Para visualizar todas as matrículas:

```
SELECT * FROM matriculas;
```

O exercício também pede uma consulta que liste os alunos e seus cursos:

```
SELECT nome, curso
FROM alunos;
```

E outra que liste os professores e as disciplinas que ministram:

```
SELECT nome, disciplina
FROM professores;
```

14. Script completo para criar o banco

Se preferir executar a criação desde o início em um único bloco, utilize o script abaixo. Ele apaga o banco `escola` caso já exista. **Atenção: isso exclui todas as tabelas e os dados que estiverem nele.** Execute somente se quiser recriar o banco do zero.

```
DROP DATABASE IF EXISTS escola;

CREATE DATABASE escola;

USE escola;

CREATE TABLE alunos (
    id INT NOT NULL AUTO_INCREMENT,
    nome VARCHAR(50),
    idade INT,
    curso VARCHAR(50),
    PRIMARY KEY (id)
);

CREATE TABLE professores (
    id INT NOT NULL AUTO_INCREMENT,
    nome VARCHAR(50),
    idade INT,
    disciplina VARCHAR(50),
    PRIMARY KEY (id)
);

CREATE TABLE matriculas (
    id INT NOT NULL AUTO_INCREMENT,
    id_aluno INT,
    id_professor INT,
    data_matricula DATE,
    PRIMARY KEY (id),
    FOREIGN KEY (id_aluno) REFERENCES alunos(id),
    FOREIGN KEY (id_professor) REFERENCES professores(id)
);

INSERT INTO alunos (nome, idade, curso) VALUES
('Ana Souza', 20, 'Ciência da Computação'),
('Bruno Lima', 22, 'Sistemas de Informação'),
('Carla Mendes', 19, 'Administração'),
('Daniel Rocha', 21, 'Ciência da Computação'),
('Eduarda Alves', 23, 'Engenharia de Software');

INSERT INTO professores (nome, idade, disciplina) VALUES
('Marcos Silva', 40, 'Banco de Dados'),
('Juliana Costa', 35, 'Programação'),
('Pedro Oliveira', 45, 'Engenharia de Software');

INSERT INTO matriculas (id_aluno, id_professor, data_matricula) VALUES
(1, 1, '2026-03-10'),
(2, 2, '2026-03-11'),
(3, 3, '2026-03-12'),
(4, 1, '2026-03-13'),
(5, 2, '2026-03-14');

SHOW TABLES;

SELECT * FROM alunos;
SELECT * FROM professores;
SELECT * FROM matriculas;
```

15. Preparar o projeto Java para conectar ao banco

A Aula 07 orienta a criação de um projeto **Java with Ant → Java Application** no NetBeans e a seguinte organização:

```
Projeto
■
■■■ drivers
■   ■■■ mysql-connector-j.jar
■
■■■ Source Packages
    ■■■ conexao
        ■■■ Conexao.java
```

O nome exato do arquivo `.jar` depende da versão do driver MySQL Connector/J que você baixar.

Adicionar o driver MySQL

1. Baixe o driver MySQL Connector/J.
2. Abra a pasta do projeto pelo NetBeans: clique com o botão direito no projeto e escolha Open in System.
3. Crie a pasta `drivers` dentro da pasta do projeto.
4. Coloque o arquivo `.jar` do driver nessa pasta.
5. No NetBeans, clique com o botão direito em Libraries.
6. Selecione Add JAR/Folder.
7. Escolha o arquivo `.jar` do MySQL Connector/J.

Criar a classe de conexão

No projeto, crie o pacote `conexao` e, dentro dele, a classe `Conexao`.

A conexão deverá utilizar os seguintes dados:

Informação Valor

--- ---

SGBD MySQL

Banco `escola`

Endereço local `localhost`

Porta padrão `3306`

Usuário de exemplo `root`

Senha A senha configurada no seu MySQL

A URL JDBC normalmente segue este formato:

```
jdbc:mysql://localhost:3306/escola
```

A senha não deve ser presumida: informe a senha real configurada no seu MySQL. O material da aula apresenta a criação da classe `Conexao` e o teste do método `getConexao()`, mas os detalhes do código exibido nas

imagens não estão disponíveis no texto extraído do PDF.

16. Organização das classes Java

A Aula 07 também apresenta a separação do projeto em pacotes:

```
Source Packages
■
■■■ conexao
■   ■■■ Conexao.java
■
■■■ beans
■   ■■■ Aluno.java
■   ■■■ Professor.java
■   ■■■ Matricula.java
■
■■■ dao
    ■■■ AlunoDAO.java
    ■■■ ProfessorDAO.java
    ■■■ MatriculaDAO.java
```

- conexao: classe responsável por estabelecer a conexão com o banco.
- beans: classes que representam as tabelas e possuem atributos correspondentes às colunas.
- dao: classes responsáveis por executar comandos SQL, como inserção, consulta, edição e exclusão.

Essa estrutura é uma organização sugerida para o projeto com base na explicação da aula. A atividade da Aula 07 pede o cadastro de alunos e professores pela interface; a Aula 08 dá continuidade com consulta e edição.

17. Verificação final

Antes de começar a programação Java, confira:

- [ ] O banco `escola` foi criado.
- [ ] O banco `escola` foi selecionado com `USE escola;`.
- [ ] As tabelas `alunos`, `professores` e `matriculas` foram criadas.
- [ ] As chaves primárias e estrangeiras foram definidas.
- [ ] Os registros de teste foram inseridos.
- [ ] Os comandos `SELECT` retornam os dados.
- [ ] O serviço MySQL está em execução.
- [ ] O driver MySQL Connector/J foi adicionado às Libraries do projeto Java.

Importante: primeiro confirme que o banco funciona diretamente no MySQL. Depois configure e teste a conexão pelo Java. Assim, se ocorrer algum erro, será mais fácil identificar se o problema está no banco, no driver ou no código de conexão.

Materiais utilizados

- Aula 07 --- Banco de dados + Java: criação do banco `escola`, tabelas, registros, consultas e preparação da conexão MySQL com Java.
- Aula 08 --- Banco de dados + Java --- Consulta, Editar e Deletar: continuidade do projeto com operações de consulta, atualização e exclusão.